

三阶段整理流程落地方案

逐个解析 → 抽取结构化信息 → 合并加工

项目	睿乐大脑 · 整理模块
版本	V1（基于代码核查）
流程	Map/Extract/Reduce
结论	阶段一已具备 · 阶段二差一步写入 · 阶段三需回写法

1
阶段已具备

1
阶段差一步

1
阶段无入口

1
处旧结论修正

三阶段整理流程落地方案

流程：逐个解析 → 抽取结构化信息 → 合并加工（Map → Extract → Reduce）前置：多条记忆整理方案.md（筛选接线）、多记忆两阶段整理方案.md（Map/Reduce 评估） 本文所有结论均带 文件:行号，凡"已定义未接线"处均明确标注。

0. 结论先行

这条流程可以用，而且三段里有两段只差接线。逐段判定：

阶段	实现状态	卡点
① 逐个解析	✅ 已具备	无「批次 / 父子任务」概念，需在应用层循环
② 抽取结构化信息	💡 提示词层已可用，回填层缺一步	<code>spec.fields</code> 声明了却从不写入 <code>output.fields</code>
③ 合并加工	❌ 无入口	成果不能当输入；需回写法或新增编排

一个需要修正的旧结论：我此前说「台账不能按渠道 / 异议类型筛选统计」——读侧错了。GET /organize/outputs?field.<key>=<value> 是通的，端到端接线完毕。真正缺的在写侧（见 §3.4）。这反而让阶段②的成本大幅下降。

1. 阶段① 逐个解析 —— ✅ 已具备

1.1 真实能力

能力	位置
提示词装配（记忆 → 提示词）	<code>buildOrganizeJobPrompt</code> ， <code>organize_workbench.go:896-960</code>
记忆正文	<code>memory.Content</code> = AI 加工后的笔记 HTML（原始转写另存 <code>metadata["transcript"]</code> ）
记忆的 AI 摘要 / 标签	写入 <code>metadata["summary"]</code> / <code>metadata["tags"]</code> ： <code>organize_memory_audio.go:456-457</code> 、 <code>organize_memory_attachments.go:566-567</code>
单条记忆喂模型	直接取 <code>Content</code> ，按 <code>[M#]</code> 编号引用

1.2 "逐个"怎么实现

整理任务是单次执行（`executeOrganizeJob`，`organize_workbench.go:617` 一次 LLM 调用出成果），没有父子 / 批次机制（`ParentJobID`、`batch_id` 全仓零命中）。

所以「逐个」= 在应用层循环：对每条记忆各建一个任务（`POST /organize/jobs`，`memory_ids: [单条]`）。

- 0 代码可行：外部脚本 / API 调用方循环即可，无需改平台。
- 若要有"一次提交、平台内部分解"的体验，需要新增批次概念（见 §4 第二步）。

1.3 一个必须处理的前置

`ensureOrganizeMemoriesReady`（`organize.go:290-297`）—— 选中的记忆里只要有一条录音还在转写 / 附件未就绪，整批任务直接报错。逐个解析是单条提交，天然规避，但循环里要把失败项单列出来。

2. 阶段② 抽取结构化信息 —— ◆ 提示词层已可用，回填缺一步

这是三段里最有价值的一段，也是最容易被误判的一段。

2.1 模板骨架已经能声明「表格列」

`spec.sections` 之外，编译器还能识别 表格占位符的列：`compiledOrganizeSection.Columns`（`organize_template_compiler.go:18-23`），编译产物写入 `markdown_contract.repeatable_placeholders`（`:246-268`）。

配套的提示词约束已经强制：

- 「表格占位符必须展开为完整的 Markdown 行，不能把整张表格压缩成一句话」（`organize_markdown_template.go:100`）
- 「保留章节顺序与标题层级」「未知数据使用 `___`，不得编造」「关键事实标注 `[M1]`」（`:96-101`）

结论：只要模板骨架里写一张表、表头就是你要的字段，模型就必须逐行填。这一步 0 代码。

2.2 但有一条硬约束：不许输出 JSON

`organizeMarkdownTemplatePrompt` 明确写死「必须输出完整的中文 Markdown，不要输出 JSON」（`organize_markdown_template.go:95`）。

所以阶段②的产物形态只能是 Markdown 正文（含表格），不能直接拿到 JSON。

2.3 平台其实有 JSON 抽取的先例

记忆入库时就在做结构化抽取：`parseOrganizeUploadAIResponse`（`organize_upload.go:425-480`）——先直接 `json.Unmarshal`，失败再截取首个 `{` 到末个 `}` 重试（容错式解析），取 `title/summary/tags/note_markdown`。

这是可复用的范式，说明"从模型输出里抽结构化字段"在本项目里不是新问题。

2.4 关键缺口：`spec.fields` 声明了，却从不写入

模板 `spec` 支持声明字段 `schema`，且有校验：`validateOrganizeTemplateSpec` 要求每个 `field` 必须带 `key` 和 `label`（`organize_admin.go:386-425`）。

但是——

环节	现状
模板声明 <code>spec.fields</code>	✅ 有（且有校验）
<code>compile</code> 生成 <code>spec.fields</code>	❌ 不生成（ <code>buildCompiledTemplateSpec</code> 只产出 <code>sections</code> + <code>markdown_contract</code> ， <code>organize_template_compiler.go:246-268</code> ）
任务产出时写 <code>output.fields</code>	❌ 硬编码 6 个系统字段（主题/模板/模板版本/涉及时间/待办数/来源数， <code>organize_workbench.go:644-652</code> ），完全不读模板声明的 <code>fields</code>
按字段筛选成果	✅ 通（见 2.5）

这就是阶段②唯一缺的一步：把模板声明的字段 `schema` 注入提示词（可用"表格表头"形式，0 代码）→ 再把模型产出的表格回填成 `output.fields`（需少量代码）。

2.5 修正：读侧筛选其实早已就绪

我此前判断「台账不能按渠道 / 异议类型筛选」——错了。链路是完整的：

- 1. 入参解析：`organizeFieldFilters` 从查询串取 `field.<key>=<value>`（`handler/organize.go:1050-1065`）
- 2. 下推：`ListOutputs` 遍历 `query.FieldFilters` 逐条下推（`repository/organize.go:399-401`）
- 3. 方言适配：`applyOrganizeJSONTextFilter`——PG 用 `fields->>key = value`，SQLite 用 `json_extract(fields, '$.key') = value`（`repository/organize.go:698-708`）

也就是说：只要 `output.fields` 里真的按模板声明写入了业务字段，`?field.异议类型=价格` 立刻可用。

注：这是等值过滤（扁平顶层 key），不支持范围 / 数值比较 / 模糊——统计仍要另算。

3. 阶段③ 合并加工 —— ❌ 无入口，需回写法

3.1 硬事实：成果不能当输入

- 输入固定取自 `organize_memories` : `executeOrganizeJob` → `ListMemoriesByIds` (`organize_workbench.go:602`)
 - 无父子 / 批次: `ParentJobID`、`batch_id` 零命中
 - Stage (`loading_memories` → `generating` → `saving_output`) 只是**展示名**，不是多阶段编排
- 一句话：平台能做「一批记忆 → 一份成果」，做不了「成果 → 再整理」。

3.2 可用的真实资产

- `organize_output_memories` 是真接线的：写入 `repository/organize.go:710-728`，读取 `fillOutputLinks:750-774`。成果能正向查到覆盖了哪些记忆 → **来源可追溯**。
- `OrganizeMemoryInput` 支持自定义 `Source` 与 `Metadata` (`types/organize.go:426-434`) → 中间稿可以打标记。

3.3 回写法 (0 行后端代码)

- | | |
|--------|---|
| ① 逐个解析 | 每条记忆各建 1 个任务 (模板 <code>T_extract</code>) → N 份成果 |
| ② 抽取 | 从每份成果取 <code>content</code> (或 <code>fields</code>) → <code>POST /organize/memories</code> 回写成一条记忆
<code>source = "merge_input", metadata = { origin_output_id, origin_memory_ids: [...] }</code> |
| ③ 合并加工 | 对这批回写记忆跑一次任务 (模板 <code>T_merge</code>) → 1 份合并成果 |

两个模式，风险完全不同：

模式	选材方式	自我复读风险
手动合并 (推荐先做)	调 API 时显式传 <code>memory_ids</code> (合并哪几条由你定)	无 —— 不依赖"最近 50 条"
定时合并	<code>config</code> 定时 → 取最近一批	有 —— 回写记忆 <code>occurred_at</code> 最新，必然被选中

3.4 三个必须处理的坑

1. **自我复读 (最严重)**：中间稿本身是记忆，定时任务会反复吃到自己 → 定时模式必须给 `ListMemories` 加排除条件。当前它只支持 `Kind` 精确 + `Keyword` (`title/content/source`) (`repository/organize.go:157-163`)，没有排除式、没有 `metadata` 过滤 → 需新增。
2. **污染记忆列表**：`kind` 有 CHECK 约束只有 4 值 (`note/record/audio/audio_card` , `000072`)，不能新增"中间稿"类型，只能靠 `source` / `metadata` 标记 + 前端过滤展示。

3. 引用链断裂：中间稿变成新记忆后 `[M#]` 编号重排，合并稿回溯不到原始记忆 → 必须在 `metadata.origin_memory_ids` 兜底存原始 ID。

3.5 Reduce 输入的纪律

Reduce 的输入必须是压缩摘要（每条结论一行 + 保留引用），不能喂子成果全文——否则 Reduce 自己会撞上同一个预算墙（总 18000 字符 / 单条 2200 / 上限 50 条，[organize_workbench.go:31-32](#)）。这是最容易踩的坑。

4. 落地路线

第一步：0 代码验证（本周）

手工跑通三段，验证质量是否真的比一次性喂进去更好：

1. 切 3~5 条记忆 → 各建 1 个任务（模板 `T_extract`）→ 3~5 份成果
2. 把成果正文回写成记忆（`POST /organize/memories` + `source` 标记）
3. 显式传 `memory_ids` 对这批复写记忆跑一次 `T_merge` → 合并成果
4. 对照实验：与“一次性把全部原文喂进去”的结果比。若两阶段没有明显更好，就不值得做第二步的编排。

第二步：接线（改动清单）

优先级	改动	位置	为什么
P1	任务产出时读模板声明的 fields 写入 <code>output.fields</code>	<code>organize_workbench.go:644-652</code>	阶段②的核心；写进去后 <code>field.X=Y</code> 筛选立刻可用
P1	从模型输出的表格回填字段（容错解析）	新增，范式参考 <code>organize_upload.go:425-480</code>	同上
P2	<code>ListMemories</code> 支持排除条件（如排除 <code>source</code> 命中项）	<code>repository/organize.go:157-163</code>	定时合并的前置
P2	整理配置支持记忆筛选（挂 <code>config.Metadata.memory_filter</code> ）	<code>organize_workbench.go:801</code> 处传参	阶段①圈定范围
P3	新增批次 / 父子任务（ <code>parent_job_id</code> + <code>phase</code> ）	迁移 + 编排	把"逐个"从应用层循环变成平台能力
P3	成果页显示「本次覆盖 N/M 条记忆」	成果详情	避免用户把"只覆盖一部分"误判成"AI漏了"

5. 参考配置

5.1 抽取模板 `T_extract`（阶段①②）

骨架里直接写表——表头就是你要的字段，模型必须逐行填：

```
# {{title}}

> {{summary}}

## 01. 线索要点
{{section_1}}

## 02. 结构化字段

| 字段 | 取值 | 依据 |
|---|---|---|
| 线索来源 | {{来源}} | {{M1}} |
| 意向课程 | {{课程}} | {{M1}} |
| 家长顾虑 | {{顾虑}} | {{M1}} |
| 下一步动作 | {{动作}} | {{M1}} |

## 03. 下一步
- [ ] {{section_3}}

### 依据
{{citations}}

**标签:** {{tags}}
```

对应 `spec` (`fields` 已支持校验) :

```
{
  "sections": ["线索要点", "结构化字段", "下一步"],
  "fields": [
    { "key": "线索来源", "label": "线索来源" },
    { "key": "意向课程", "label": "意向课程" },
    { "key": "家长顾虑", "label": "家长顾虑" },
    { "key": "下一步动作", "label": "下一步动作" }
  ],
  "markdown_contract": { "missing_value": "___" }
}
```

建单可直接用 `compile` (`POST /system/admin/organize/templates/compile`) : 给一段 Markdown, 自动生成 key / name / scene / sections.

5.2 合并模板 `T_merge` (阶段③)

章节固定为「已排除项 / 重复项 / 合并结论 / 待确认」，并在指令里**显式要求列出被剔除的条目**——因为提示词约束是"不得编造"，排除动作必须留痕。

6. 一句话回答

这条流程可以采用：阶段①已具备（循环提交即可）、阶段②**只差一次字段回填**（提示词层已成、读侧筛选已就绪）、阶段③**需要回写法过渡**。

最该先做的一件事：手工跑「切 3 批 → 3 份成果 → 回写 → 合并」的对照实验。它是决定要不要投入方案编排的**唯一依据**。